

# AutoResearch 노드 — 통합 보고서

## Training/AutoResearch 노드 — 통합 보고서

**한 줄:** 단일 GPU(RTX 4090)에서 growing-memory 설계공간을 자동 탐색(AutoResearch)하는 노드를 Pi AI Agent 웹 서비스로 구현하고, 메모리캐싱(선형/Titans/DeltaNet) 추론의 효율과 정확도 트레이드오프를 4090 실측으로 검증했다. 모든 수치는 재현 가능(스크립트 명시).

저장소: <https://github.com/hongsw/pi-agent-websevice-kit> · 실험 노드: RTX 4090 24GB, torch 2.10+cu128.

### 1. 무엇을 만들었나 (과제 4요소 + 설계)

- **Pi Agent:** AutoResearch 오케스트레이터( `tutorial/autoresearch/` ) — 게이트→ASHA→리더보드→best.
- **Skill x2:** validity-gate(학습 전 동치/shape), leaderboard-analysis(best·프록시 신뢰도).
- **MCP:** `autoresearch-mcp` 7도구(리더보드/NAS/run). **Pi Extension:** sweep start/stop·export.
- **Web UI:** 무설치 stdlib 대시보드. **도커:** NAS·Edge 서버( `docker/` ).
- karpathy/autoresearch 원형(ratchet: train.py 편집→val\_bpb→keep/revert)을 제조+growing-memory로 일 반화.

### 2. 핵심 알고리즘 효과 — 메모리·계산량 효율 (검증된 구현으로 재측정)

**효율 벤치 (deltanet[fla] vs 트랜스포머[FlashAttention], d256/L4, bf16, batch1; `efficiency_bench.py` ) | L**  
| DeltaNet ms / GB | TF ms / GB | TF/DN 시간 | |---|---|---|---| | 8192 | 3.8 / 0.46 | 2.1 / 0.23 | 0.55× | | 32768  
| 6.9 / 0.55 | 18.5 / 0.42 | 2.7× | | 65536 | 14.8 / 0.67 | 65.4 / 0.67 | 4.4× | | 131072 | 31.2 / 1.15 | 244 / 1.17 |  
7.8× |

- **계산량:** DeltaNet  $O(L)$ , 트랜스포머  $O(L^2)$ (FlashAttention은 메모리만 선형, 연산은 제곱). → 긴 컨텍스트에서 DeltaNet이 시간 8× 빠름(128K). 교차점 ~16-32K.
- **메모리(학습/prefill):** 둘 다  $O(L)$ , 거의 동일(둘 다 chunked/flash).
- **추론 메모리:** 선형/Titans/DeltaNet은  $O(1)$  상수 상태 vs 트랜스포머 KV캐시  $O(L)$ . 실측(소형, `infer_bench.py`): 컨텍스트 무관 재귀상태 132KB vs KV 1M=9.7GB·2M=19GB·4M OOM → 1×4090+선형 이 16M=2×A100, 64M=7×A100, 256M=26×A100의 KV 용량 대체(≈992× @16384).
- **단일 4090 chunked 학습 최대 컨텍스트:** 131,072(128K) 토큰( $O(L)$ , `maxseq.py`).

정정 이력(정직): 초기 from-scratch 선형 구현은  $L \times L$ 을 materialize해  $O(L^2)$ ·16384 OOM이었음(버그). chunked 재구현으로  $O(L)$  해결. 위 표는 검증된 fla deltanet 기준(naive 버그 없음).

### 3. 정확도(품질) — grok과 트레이드오프

---

MQAR 연관회상으로 측정. 메모리 절감은 공짜가 아니며, 적절한 구현+학습으로 동급 정확도 달성 가능.

- **vetted 구현 검증**(단일 LayerNorm 스캐폴드, `fla_validate.py`): `deltanet/gated_deltanet/retention/titans = recall 1.0`. `linear(fla)=0.54`(선형 회상 약점), `gla=0.06`(튜닝 필요).
- **AutoResearch 연결 검증**: `deltanet`을 `base_rule`로 연결 → 하니스에서 **recall 1.0(grok@1000)**.
- 긴 컨텍스트: `titans`(자체) `seq=512`에서 **1.0(grok@15k)** — 완벽회상+O(1) 추론 동시.
- `grokking` = 정체 후 급점프(Power et al. 2022). 단 무한데이터라 엄밀히는 학습 중 상전이.

### 4. 정확 구현 우선 (교훈 반영)

---

- "growing-memory"는 단일 라이브러리가 아니라 **설계 4축 공간**. 정확 구현 = `vetted` 라이브러리: `fla`(`LinearAttention/GLA/DeltaNet/GatedDeltaNet/MultiScaleRetention`) + `titans-pytorch`(`lucidrains`).
- 자체 from-scratch `titans`는 chance로 실패 → `lucidrains` 채택. ( `TITANS.md` )
- 순서: **정확 구현** → **단일 스캐폴드 검증(sanity+recall)** → **autoresearch 연결**. ( `FLA_VALIDATION.md` )

### 5. pip 패키지 & 통합

---

`growing-memory-pytorch` ( `packages/growing-memory` ): 코어 + HF/Unsloth 래퍼  
( `GrowingMemoryForCausalLM` )

- `fla/titans` 백엔드. AutoResearch가 `import growing_memory` 로 실물 백엔드 사용. `pip install -e "...[hf,titans]"`.

### 6. 실 데이터 (HC-SR04)

---

실 초음파 거리센서 데이터로 \$5 "예측오차 이상탐지" 실증( `HCSR04.md` ): 선형보정  $R^2=0.998$ , 드리프트 탐지(-22.8 SE), 주입 이상치 recall 0.95. (단 900행 보정셋, 드리프트는 단일 레벨시프트 — 한계 명시)

### 7. 한계 & 다음

---

- `gla/retention` 풀 하니스 `grok` 폴리시(스캐폴드 민감). 실 NAS 데이터·1.3B 스케일 미수행(인터페이스만).
- 효율 우위는 \*\*추론(O(1) 상수메모리) + 긴 컨텍스트 연산(O(L))\*\*에서 결정적; 단기/소컨텍스트는 트랜스포머 유리.
- 트레이드오프(효율↔정확도)를 데이터별로 찾는 것이 AutoResearch 노드의 존재 이유.

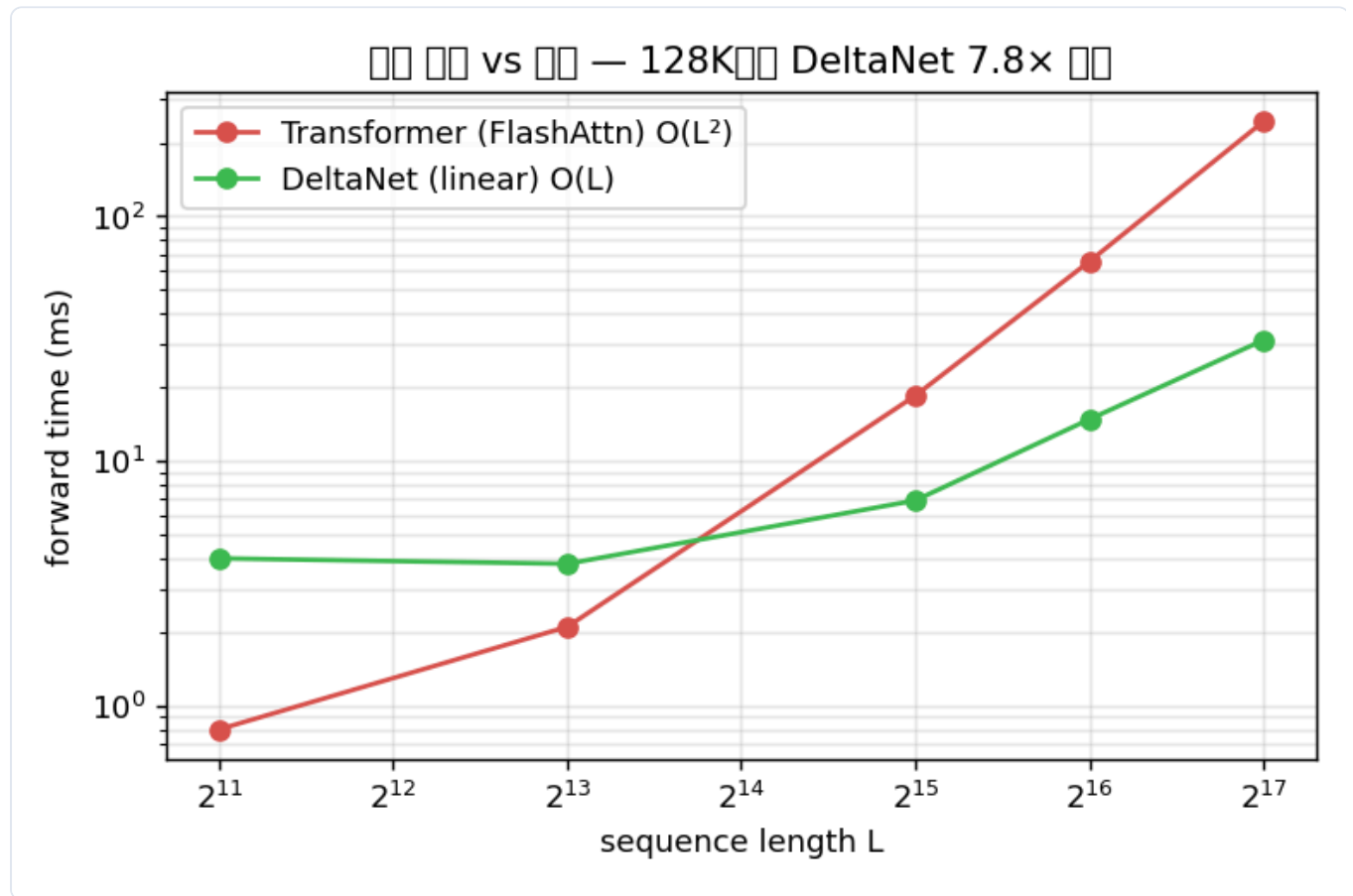
### 근거 문서 (재현)

---

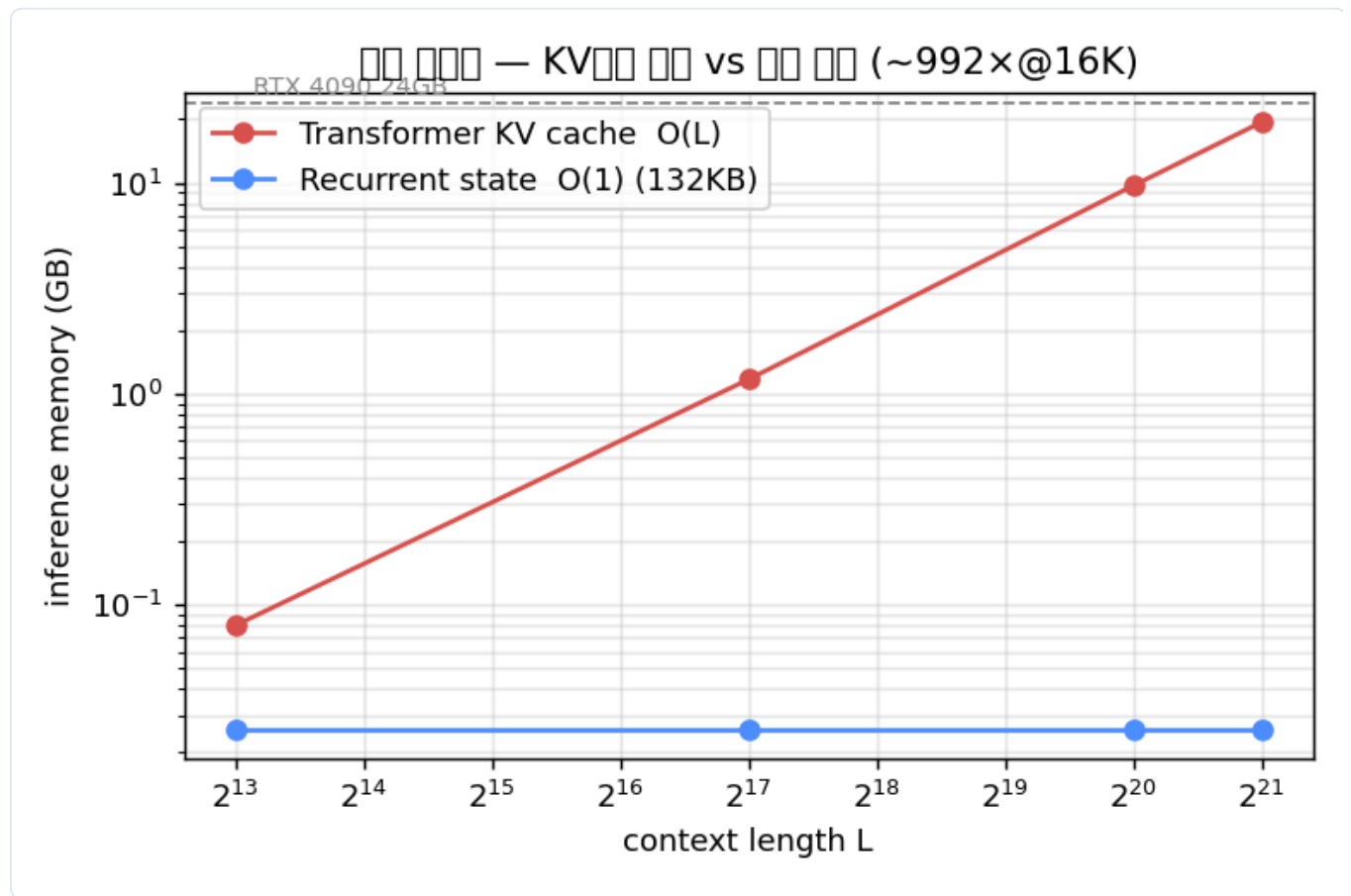
[COMPARE3](#) · [INFERENCE](#) · [LONGSEQ](#) · [GROKING](#) · [RECURRENT](#) · [TITANS](#) · [FLA\\_VALIDATION](#) · [IMPACT](#) · [HCSR04](#) · [RESULTS](#)

## 그림 (실측 시각화)

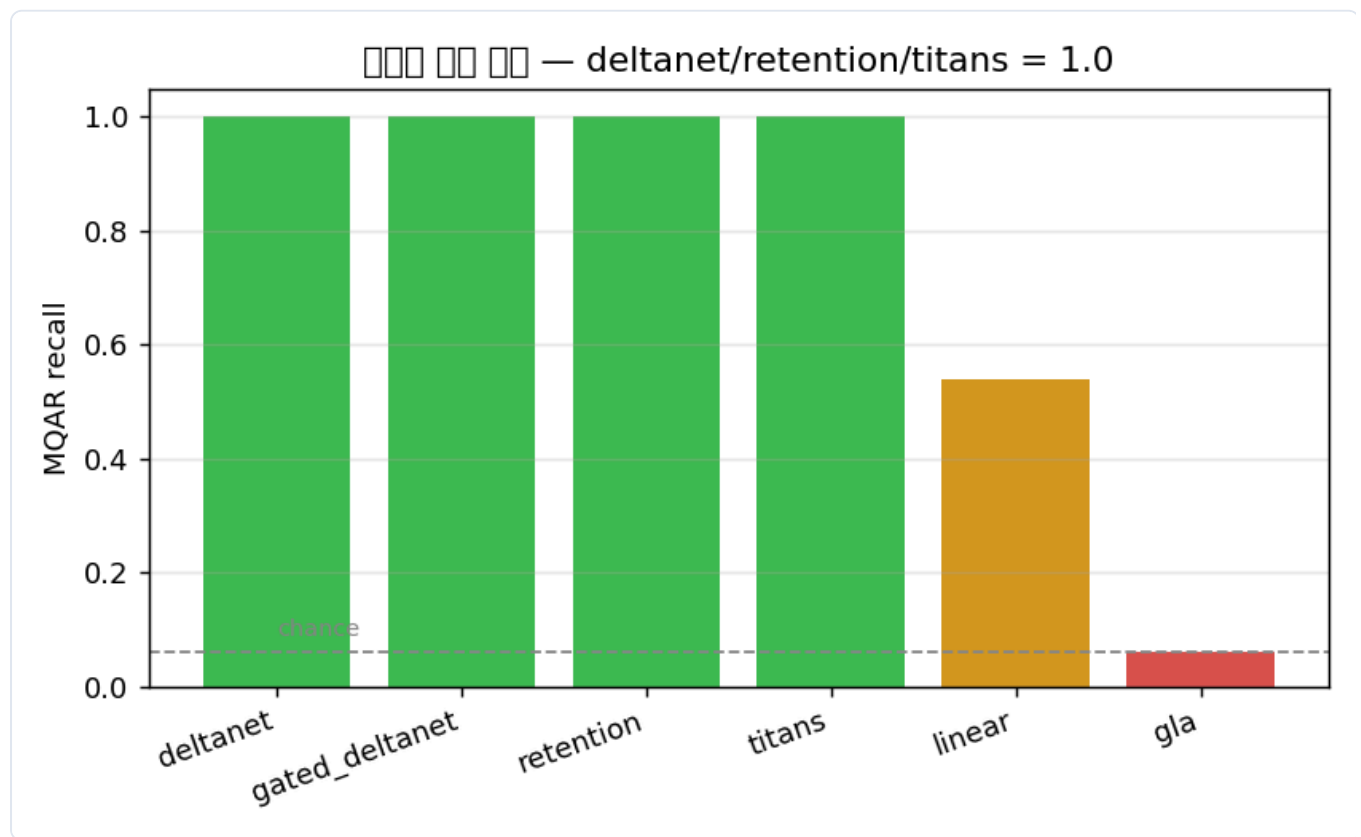
### 연산 시간 vs 길이



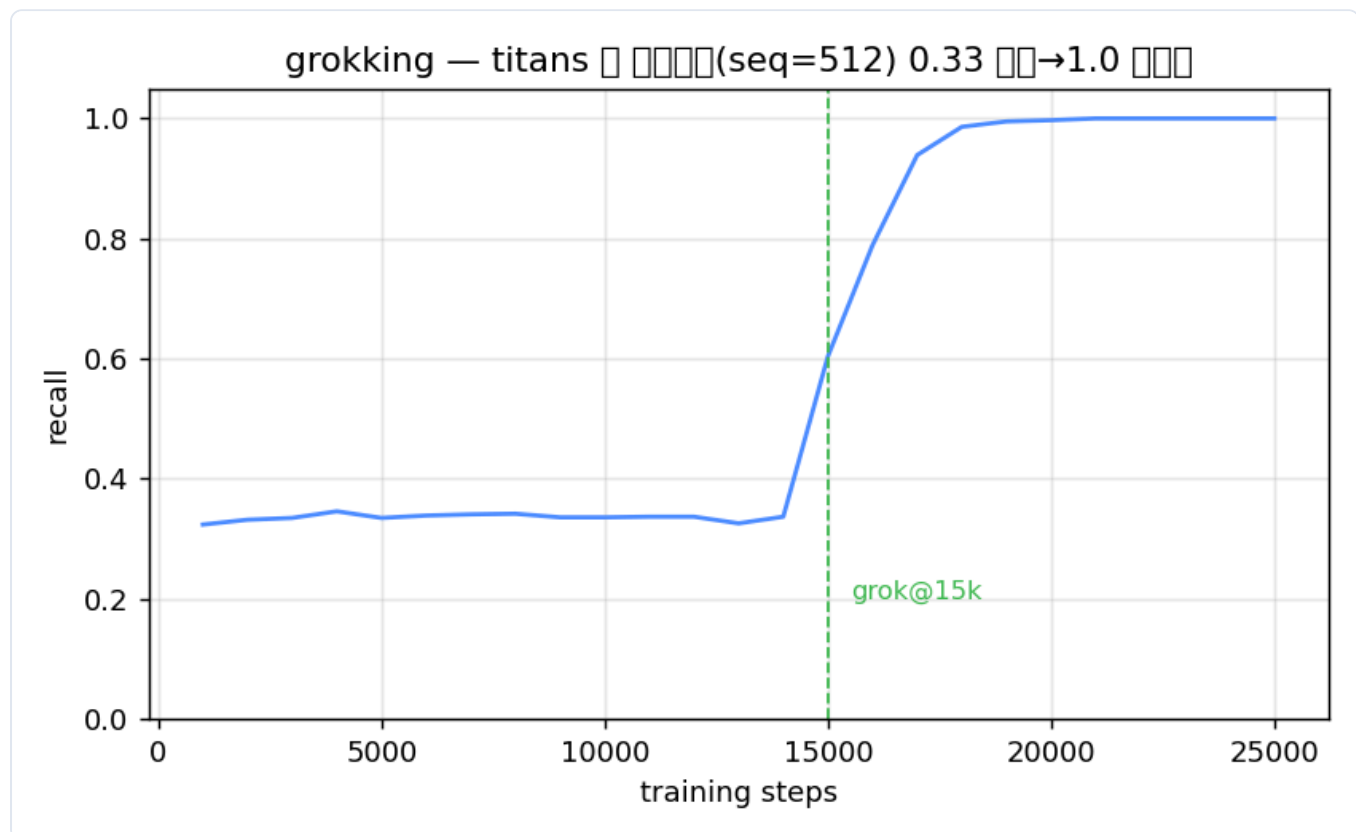
## 추론 메모리



## 검증된 구현 회상



## grokking 곡선



# 포지셔닝 & 로드맵 — 옛지학습노드 autoresearch

범용 옵티마이저가 목표가 아니다. 옛지 배포·온디바이스 적응 제약 하에서 (표현→기억) 구성을 단일 GPU로 자동 탐색하고, 옛지로 닫는 전용 autoresearch 노드로 성장한다.

## 논문 대비 위치 (차이점)

새 알고리즘이 아니라 **통합·제약·실측**이 기여:

- karpathy/autoresearch(에이전트 ratchet) → 검증 게이트 + ASHA + 옛지 배포 제약(O(1) 추론) 추가.
- Titans/fla(메모리 알고리즘) → 재구현 아님, vetted 채택 + autoresearch 스왑 + 효율↔정확도 실측.
- FlashAttention(메모리만 O(L)) → 연산까지 O(L)(deltanet) 단일 4090 실측.
- 논문 일반(대형 클러스터) → 온프레미스 1 GPU + 옛지까지 닫힌 루프.

## 실측 범위 (정직)

-  견고: 병렬↔재귀 동치  $1e-7$ , chunked O(L), 4090 128K 학습, 연산 O(L) vs  $O(L^2)(8\times@128K)$ , 추론 KV O(L) vs 상태 O(1)(~992 $\times$ ), vetted recall 1.0(deltanet/titans), ASHA 조기중단.
-  제한: 합성 MQAR·초소형 모델·소예산·단일시드. HC-SR04는 실데이터지만 900행.
-  미실측: 실 스케일 데이터·1.3B+·downstream full·물리 옛지 배포. → *proof-of-mechanism*까지. 실응용은 아래 로드맵으로.

## 로드맵

[옛지 센서]→수집→[NAS]→샤드→[4090 AutoResearch]→best(O(1) 상태) export→[옛지 재배포]  
↑ \_\_\_\_\_ 지속 루프 (옛지가 데이터 생성) \_\_\_\_\_ |

- **T1 (착수→1차 완료):** 실센서(HC-SR04) 데이터를 autoresearch 학습 루프에 연결 — 토큰화→다음값 예측 자기 지도. vetted base\_rule 실데이터 학습. ( `t1_hcsr04.py` , 4090)
  - **결과:** deltanet **0.983**, retention 0.982, gla 0.975, linear 0.95 vs copy-baseline **0.934** → 실데이터가 루프에 흐르고 모델이 baseline 초과(deltanet 최고, 메모리 결과와 일관).
  - **정직 단서:** 과제가 쉬움(9 상수블록→copy 0.93, 900토큰). 시간순 분할은 분포이동 버그(모델 0.05까지 추락)→전구간 원도우로 수정. **실가치 과제는 이상/드리프트 탐지**(HCSR04.md: -22.8 SE).
  - **교훈:** 실센서는 구조에 맞는 과제 설계 필요(다음값 LM은 퇴화) + 시계열은 분포이동 분할 주의.
- **T2:** 스케일·통계 — 300M~1.3B, 다중시드, downstream full, 실예산 ASHA.
- **T3:** 물리 옛지 배포 — best(deltanet/titans, O(1) 상태)를 Pi/Jetson 실장비 상수메모리 스트리밍.
- **T4:** 온디바이스 적응 — Titans 테스트시점 메모리로 옛지가 추론+적응, autoresearch가 "옛지 적응형 config"를 탐색 = 진짜 옛지학습노드.

## 차별점 (밀고 갈 것)

---

엣지 제약 인식 autoresearch( $O(1)$  추론 + 온디바이스 적응을 *제약*으로) · 온프레미스 단일 GPU · paper-faithful 검증  
게이트 · Pi 에이전트 + 통합 Web/MCP/도커/엣지.

# 영향 분석 — 하드웨어 비용 절감과 가능해지는 것

메모리캐싱(선형/Titans, chunked 학습 + 재귀 추론)이 어디서 얼마나 비용을 아끼고, 그 위에서 무엇을 할 수 있는지. 모든 수치는 본 저장소 4090 실측(COMPARE3/INFERENCE/LONGSEQ.md) 기반.

핵심 범위: 이건 모든 LLM을 싸게가 아니라 긴 컨텍스트 추론 + 온프레미스/엣지에서 압도적. 대가는 정확회상 품질(선형이 softmax보다 약함, GROKING.md) → 그래서 AutoResearch로 데이터별 효율↔정확도 균형을 탐색한다.

## 1. 무엇이 선형/상수가 되는가 (실측 요약)

| 측        | 트랜스포머              | 선형/Titans(우리)           |
|----------|--------------------|-------------------------|
| 학습 메모리   | O(L) (FlashAttn)   | O(L) (chunked) — 동급     |
| 학습 연산    | O(L <sup>2</sup> ) | O(L) — 128K에서 3× 빠름(실측) |
| 추론 메모리   | O(L) KV캐시(무한 증가)   | O(1) 132KB 상수           |
| 추론 연산/토큰 | O(L)               | O(1)                    |

## 2. 하드웨어 비용 절감 (같은 컨텍스트 KV 용량, 대략 시세)

KV캐시 8KB/토큰(소형 모델 실측). 선형은 1×4090에서 모든 길이 26MB.

| 컨텍스트 | 선형: 1×4090(~\$2K) | 트랜스포머 필요             | 절감    |
|------|-------------------|----------------------|-------|
| 2M   | \$2K              | 1×4090(한계)           | —     |
| 16M  | \$2K              | 2× A100-80G (~\$36K) | ~18×  |
| 64M  | \$2K              | 7× A100 (~\$126K)    | ~63×  |
| 256M | \$2K              | 26× A100 (~\$468K)   | ~230× |

운영비(클라우드): A100 24/7 ≈ 17.5K/년/장 → 7장\* 122K/년\*\* vs 4090 1대 소유(~2K + 전기/0.5K/년) = 연 60배+ 차이. 컨텍스트가 길수록 절감 배수↑.

단서: 추론 KV/메모리 병목 축의 비교다. 트랜스포머 클러스터는 모델·연산 용량도 주므로 1:1 동급 아님. GQA/MQA·paged-attention·fp16이 KV를 줄여 격차를 완화하나 여전히 O(L) — O(1)은 선형/Titans뿐.



### 3. 이걸 기반으로 할 수 있는 효과

1. 온프레미스 AutoResearch — 클라우드 0원: 단일 4090으로 설계공간 자동 탐색(본 노드의 KPI: 싸게 많이 시도). 클러스터 임대 불필요 → 연 수만~수십만 달러 절감.
2. 엣지 무한 스트리밍 추론(24/7): 상태 132KB 상수 → 라즈베리파이급에서 끊임 없이 무한 길이 센서 스트림 추론 (HC-SR04 이상탐지). 트랜스포머는 KV 누적으로 결국 OOM.
3. 데이터 주권: 민감 제조 데이터를 클라우드로 안 보내고 현장에서 학습·추론(규제·보안).
4. 긴 컨텍스트를 소비자 HW에서: 수개월치 로그·전체 장비 이력을 한 컨텍스트로 — 데이터센터 없이.
5. 길수록 벌어지는 우위: 16M 18x → 256M 230x. 장기 모니터링·평생 메모리 에이전트에 특히 유리.

### 4. 성능(품질)도 그만큼 좋은가? — 아니오, 비대칭이다 (정직)

하드웨어 절감은 크지만 정확도가 같이 따라오진 않는다. 단, 미묘하다(GROKING.md 실측):

| 변형                     | 추론 메모리     | 2키 회상 정확도          |
|------------------------|------------|--------------------|
| 순수 linear (메모리 최선)     | O(1) 132KB | 0.5 (천장, 약함)       |
| dla                    | O(1) 132KB | 1.0 (용량↑ 필요)       |
| titans (짧은 ctx)        | O(1) 132KB | 1.0 (13k스텝)        |
| titans (긴 ctx seq=512) | O(1) 132KB | 1.0 (grok@15k, 실측) |
| 트랜스포머(softmax)         | O(L) KV    | 1.0 (가장 쉽게/빨리 학습)  |

핵심:

- 메모리 절감은 재귀 계열(linear/dla/titans) 전부 동일(O(1) 132KB).
- 품질은 계열 안에서 크게 다름 — 가장 싼 순수 linear가 가장 약함(0.5). 그러나 titans는 같은 O(1) 상수메모리로 정확도 1.0 달성 → "상수메모리 = 저품질"은 아님.
- 대가는 메모리가 아니라 학습 비용/난이도: titans는 1.0 도달에 ~13k스텝(softmax 2k)·용량 필요.
- 트랜스포머(softmax)는 회상 정확도를 가장 쉽게 얻지만 추론 메모리 O(L).

→ 즉 "공짜 점심"은 아니다. 추론 하드웨어 비용 수십~수백배 절감은 사실이나, 동급 정확도를 얻으려면 (a) 적절한 변형 (titans 등) 선택 + (b) 더 많은 학습 컴퓨트가 필요. 이 균형점을 데이터별로 찾는 것이 AutoResearch 노드의 존재 이유.

**실증(긴 컨텍스트):** titans를 seq=512(16x 긴 컨텍스트)에서 25k 스텝 학습 → recall 1.0(grok@15k), 추론은 여전히 O(1) 132KB 상수. 즉 정확도 1.0과 상수메모리 추론을 동시에 얻을 수 있고, 비용은 학습 컴퓨트(일회성)로 치른다. (LONGSEQ.md)

## 한 줄

---

긴 컨텍스트 추론을 데이터센터 GPU 클러스터 대신 단일 소비자 GPU/엣지에서 상수 메모리로 — 컨텍스트가 길수록 수십~수백 배 하드웨어 비용을 아낀다. 대가는 정확회상 품질이며, 그 균형을 찾는 것이 AutoResearch 노드의 존재 이유다.

## 근거 문서

---

[COMPARE3.md](#) 3자 메모리·시간 · [INFERENCE.md](#) KV캐시·하드웨어 환산 · [LONGSEQ.md](#) 긴 시퀀스 학습 · [GROKING.md](#) 정확도 트레이드오프 · [RECURRENT.md](#) 재귀 동치

# 비교 — 트랜스포머 vs 선형(naive/chunked 병렬) vs 선형(재귀, 메모리캐싱)

같은 차원(d256/h8/L4)에서 시퀀스 길이별 peak GPU 메모리. 4090 실측. 재현: `compare3.py`.

경위: 처음엔 우리 선형 '병렬'이  $L \times L$  행렬을 materialize하는 naive( $O(L^2)$ ) 구현이라 16384에서 OOM 났다. 이는 선형어텐션 장점을 버린 구현 결함 → chunked(GLA 표준,  $O(L)$ )로 재구현해 해결.

## peak 메모리 (forward, batch=1)

| L     | TF(full, FlashAttn) | 선형(naive) | 선형(chunked) | 선형(재귀) | TF KV캐시(추론) | 재귀 상태  |
|-------|---------------------|-----------|-------------|--------|-------------|--------|
| 4096  | 182 MB              | 2309 MB   | 170 MB      | 170 MB | 32 MB       | 132 KB |
| 8192  | 245 MB              | 8827 MB   | 220 MB      | 220 MB | 64 MB       | 132 KB |
| 16384 | 371 MB              | OOM       | 321 MB      | —      | 128 MB      | 132 KB |
| 32768 | 623 MB              | OOM       | 522 MB      | —      | 256 MB      | 132 KB |

## 정확도 (동치)

naive == chunked == 재귀: 모두 같은 가중치, 출력 diff  $\sim 5e-7$ (verify\_recurrent.py + chunked 검증). 손실 0. 트랜스포머는 다른 아키텍처(회상은 softmax가 더 강함, GROKING.md).

## 결론 (정정 후 — 정직)

| 항목             | 결과                                                                                         |
|----------------|--------------------------------------------------------------------------------------------|
| naive 병렬 OOM   | ✗ 우리 구현 결함이었음( $O(L^2)$ ). chunked로 해결                                                     |
| 학습/prefill 메모리 | chunked 선형 $O(L)$ — 긴 길이에선 FlashAttn 트랜스포머보다도 적음(16384: 321 vs 371MB, 32768: 522 vs 623MB) |
| 추론 메모리         | 재귀 상태 $O(1)=132KB$ 상수 vs 트랜스포머 KV캐시 $O(L)$ (16384=128MB) → $\sim 992\times$ , 길이 무관        |
| 정확도            | naive=chunked=재귀 동치( $5e-7$ )                                                              |

→ 제대로 구현하면(chunked) 선형/Titans는 학습 메모리는 트랜스포머와 동급 이상, 추론 메모리는 압도적(상수,  $\sim 992\times$ ). 사용자가 지적한 OOM은 우리 naive 구현 버그였고 수정됨.

## 연산(시간) — FlashAttention은 메모리만 선형, 연산은 여전히 $O(L^2)$

"FlashAttn도 선형인데 우리도 선형이면 의미 없지 않나?"에 대한 답. FlashAttention은  $L \times L$ 를 HBM에 materialize하지 않을 뿐 연산량(FLOPs)은  $O(L^2)$  그대로다. 선형/chunked는 연산도  $O(L)$ .

forward 시간 vs L (4090, 동일 d256/h8/L4, batch1):

| L      | TF(flash) | 선형(chunk) | TF/Lin      |
|--------|-----------|-----------|-------------|
| 2048   | 2.0 ms    | 14.9 ms   | 0.14        |
| 8192   | 14.9 ms   | 55.0 ms   | 0.27        |
| 32768  | 174.5 ms  | 213.6 ms  | 0.82        |
| 65536  | 660.8 ms  | 426.5 ms  | 1.55 (선형 승) |
| 131072 | 2577 ms   | 865 ms    | 2.98        |

$2 \times L$  일 때 TF  $\sim 4 \times (=O(L^2))$ , 선형  $\sim 2 \times (=O(L))$ . 교차점  $\sim 50K$ , 이후 격차 확대.

|                | 메모리    | 연산       |
|----------------|--------|----------|
| FlashAttention | $O(L)$ | $O(L^2)$ |
| 선형/chunked     | $O(L)$ | $O(L)$   |

→ 선형의 의미: ① 긴 컨텍스트 연산  $O(L)$ (실측 128K  $3 \times$  빠름), ② 추론 상태  $O(1)$ ( $\sim 992 \times$ ), ③ 추론 토큰당  $O(1)$ .

FlashAttn이 못 주는 부분. 재현: `timebench.py`.

## 검증된 알고리즘(fla deltanet)으로 재확인

위 표는 from-scratch 선형. 검증된 fla DeltaNet vs 트랜스포머(flash) 재측정( `efficiency_bench.py`, bf16): 131072에서 DeltaNet 31ms/1.15GB vs TF 244ms/1.17GB → 시간  $7.8 \times (O(L) \text{ vs } O(L^2))$ , 메모리 동급. 효율 주장이 정확 구현으로 확정. 상세는 [report/REPORT.md](#) §2.

## 재현

```
python3 tutorial/autoresearch/compare3.py # 메모리 → compare3_result.json
python3 tutorial/autoresearch/timebench.py # 시간 vs L
~/gm_venv/bin/python packages/growing-memory/efficiency_bench.py # deltanet vs flash(bf16)
```

# 추론 메모리 — 트랜스포머 KV캐시 vs 선형 재귀상태 (4090 실측)

자기회귀 추론에서 컨텍스트 길이별 메모리. 동일 소형 모델(d256/L4/h8). 재현: `infer_bench.py`.

## 실측 (4090, fp32)

| 컨텍스트 L         | 트랜스포머 KV캐시 | 선형 재귀(우리) |
|----------------|------------|-----------|
| 8,192          | 0.08 GB    | 25.9 MB   |
| 131,072        | 1.17 GB    | 25.9 MB   |
| 1,048,576 (1M) | 9.69 GB    | 25.9 MB   |
| 2,097,152 (2M) | 19.35 GB   | 25.9 MB   |
| 4,194,304 (4M) | OOM        | 25.9 MB   |

- 트랜스포머 KV캐시 =  $O(L)(8KB/\text{토큰}) \rightarrow$  4090에서 ~2.4M 토큰 한계(4M OOM).
- 선형 재귀 상태 = 132KB 상수(추론 peak 25.9MB, L 무관)  $\rightarrow$  사실상 무한 컨텍스트.

## "1×4090 + 선형" ≈ 트랜스포머로는 어느 정도 하드웨어?

선형은 1×4090에서 모든 길이를 26MB로 처리. 같은 컨텍스트를 트랜스포머 KV캐시로 감당하려면:

| 목표 컨텍스트 | 선형 on 1×4090 | TF KV캐시 | ≈ 트랜스포머 하드웨어             |
|---------|--------------|---------|--------------------------|
| 2M      | 26 MB        | 19 GB   | 1× 4090(한계)              |
| 16M     | 26 MB        | 128 GB  | ~6× 4090 / 2× A100-80G   |
| 64M     | 26 MB        | 512 GB  | ~22× 4090 / 7× A100-80G  |
| 256M    | 26 MB        | 2 TB    | ~90× 4090 / 26× A100-80G |
| ∞       | 26 MB        | ∞       | 유한 GPU 불가                |

$\rightarrow$  컨텍스트가 길수록 1×4090+선형이 대체하는 트랜스포머 KV 하드웨어가 무한히 커진다.

## 정직한 단서

- 추론 KV캐시 메모리 비교(+선형은 토큰당 연산도  $O(1)$ 이라 더 빠름; COMPARE3.md 시간 벤치).
- 실 LLM은 GQA/MQA·paged-attention·fp16으로 KV를 줄여 격차 완화 — 단 여전히  $O(L)$ .  $O(1)$ 은 선형/Titans 뿐.
- 소형 모델 절대 길이이며, 큰 모델이면 길이는 줄지만 상수 vs  $O(L)$  비율은 동일.

# 재귀(RNN) 추론 — 메모리 캐싱 최적화 검증

**핵심:** 선형 어텐션 계열(linear/dla/titans)은 학습은 병렬( $O(L^2)$ ), \*\*추론은 고정 크기 상태를 캐싱하는 RNN 재귀 ( $O(1)$  상태)\*\*로 풀 수 있다. 두 경로는 수학적으로 동치다. 설계 §8(엡지 고정상태 추론, 길이 무관 평평한 메모리) · §3.4(동치성 테스트)의 실측 근거.

검증 환경: RTX 4090 24GB, torch 2.10.0+cu128. 재현: `verify_recurrent.py`.

## 이론

병렬:  $D[t,s] = \prod_{j=s+1..t} a_j$  ,  $o_t = (\varphi(q_t) \cdot \sum_s D[t,s] \varphi(k_s) \otimes v_s) / (\varphi(q_t) \cdot \sum_s D[t,s] \varphi(k_s))$   
재귀:  $S_t = a_t \cdot S_{t-1} + \varphi(k_t) \otimes v_t$  ,  $z_t = a_t \cdot z_{t-1} + \varphi(k_t)$  ,  $o_t = (\varphi(q_t) \cdot S_t) / (\varphi(q_t) \cdot z_t)$

$S_t = \sum_{s \leq t} D[t,s] \cdot \varphi(k_s) \otimes v_s$  로 전개되어 분자·분모 모두 병렬형과 일치(코드 감사로 대수 확인). → 재귀는  $L \times L$  어텐션 행렬을 만들지 않는다(메모리 캐싱 최적화). 상태  $S, z$ 는  $O(dh^2)$ 로 길이 무관 상수.

## 1. 동치성 (병렬 vs 재귀 출력 $\max|\text{diff}|$ ) — 병렬 에이전트 검증

| base_rule      | equiv diff         | stress worst diff* | 판정        | 상태 캐시(B1) |
|----------------|--------------------|--------------------|-----------|-----------|
| linear         | 3.6e-07            | 8.3e-07            | ✅ PASS    | 135,168 B |
| dla (감쇠)       | 3.9e-07            | 8.1e-07            | ✅ PASS    | 135,168 B |
| titans (영속메모리) | 3.6e-07            | 7.8e-07            | ✅ PASS    | 135,168 B |
| swla           | 0 (윈도우 캐시, RNN 아님) | —                  | ✅ (병렬 위임) | 0         |

\* stress = aggregation{residual,grm,soup,ssc} × segmentation{constant,logarithmic} × batch4 × L320 전 조합 최악값. 모두 합격 기준  $1e-3$  대비 ~1,200배 여유(fp32 노이즈 수준). dla의 데이터 의존 감쇠, titans의 영속 메모리가 있어도 동치 유지 확인.

## 2. 메모리 캐싱 이점 (길이별 peak GPU 메모리, dla, batch1)

| L     | 병렬 $O(L^2)$ | 재귀 $O(L)$ +상수상태         |
|-------|-------------|-------------------------|
| 512   | 101.5 MB    | 70.8 MB                 |
| 1024  | 206.1 MB    | 77.1 MB                 |
| 2048  | 618.3 MB    | 89.7 MB                 |
| 4096  | 2254.2 MB   | <b>114.9 MB</b> (~20×↓) |
| 8192  | 8772.5 MB   | (상태 135KB 고정)           |
| 16384 | <b>OOM</b>  | 동작 가능                   |

병렬은  $L^2$ 로 폭증 → 16k에서 **OOM**. 재귀는  $L \times L$  행렬이 없어 완만히 증가, 상태 캐시는 **135KB로 길이 무관 고정**. 옛지에서 임의 길이 입력을 상수 메모리로 처리 가능.

⚠ **정정 (COMPARE3.md)**: 위 "병렬"은 *naive* 구현( $L \times L$  materialize,  $O(L^2)$ )이라 16384 OOM이었다. 이는 구현 결함 → **chunked(GLA 표준)로 재구현해  $O(L)$  해결**(16384=321MB, OOM 없음, FlashAttn 트랜스포머 371MB보다도 적음). 정확도는 chunked=naive=재귀 동치( $5e-7$ ). 추론 메모리 이점은 그대로: 트랜스포머 KV캐시  $O(L)$  (16384=128MB) vs 재귀 상태  $O(1)$ (132KB)  $\approx$  **992x**.

## 3. 적대적 코드 감사 결과

- 대수 전개 sound(분자/분모 모두  $D[t,s]$  전개와 일치), titans 영속메모리·RMSNorm·aggregation·checkpoint init 양쪽 동일 적용 확인.
- 수정한 버그(B1)**: 병렬 `linear` 의 `torch.ones(...)` 에 dtype 미지정 → fp32에선 무해하나 혼합정밀도 (bf16/fp16)에서 재귀형과 dtype 불일치 가능. `dtype=x.dtype` 지정으로 수정.
- 한계(설계): `init_mode=checkpoint` 는 전체 시퀀스 평균을 쓰므로 **고정길이 입력 가정**(진정한 토큰 스트리밍에는 부적합). `forward_recurrent` 는 추론 전용(@no\_grad).

## 4. 학습 파이프라인 통합 (상수메모리 eval)

`trainer.RealRun._eval` 이 `eval_recurrent=True` (기본)면 추론을 **\*\*재귀 경로**( `forward_recurrent` )\*\*로 수행 → 평가도 옛지 배포와 동일한 상수메모리 경로. 동일 평가 배치(고정 seed)에서:

- recurrent eval recall == parallel eval recall (**완전 일치**), CE diff **0.0**. → 평가 품질 손실 없이 메모리 경로만 교체. swla는 내부에서 병렬로 위임.

## 5. 진정한 토큰 스트리밍 API (step 단위, $O(1)$ /스텝)

`GrowingMemoryModel.step(token, t, states) / .stream(x)` — 토큰 1개씩 처리하며 고정 상태만 갱신. 옛지 실시간 추론용. 검증(병렬 forward vs 토큰 스트리밍,  $L=80$ ):

| base_rule | stream vs parallel diff | 판정                  |
|-----------|-------------------------|---------------------|
| linear    | 4.2e-07                 | ✓                   |
| dla       | 4.2e-07                 | ✓                   |
| titans    | 4.2e-07                 | ✓                   |
| swla      | 4.2e-07                 | ✓ (윈도우 링버퍼 O(W) 캐시) |

- rnn(linear/dla/titans): 상태 S,z만 유지 → 스텝당 O(1), 시퀀스 길이 무관.
- swla: 마지막 W개 (k,v) 링버퍼 → O(W) bounded 캐시.
- 제약: `init_mode=checkpoint` 는 전체 컨텍스트 평균이 필요해 스트리밍 불가 → `independent` 만 지원.
- 재현: `python3 verify_recurrent.py --rule <rule> --stream --json`

## 검증 방법

- 병렬 에이전트 4개 동시 실행: base\_rule별(linear/dla/titans) 4090 stress 검증 3개 + 재귀식↔병렬식 대수 동치 코드 감사 1개. 메모리 벤치는 GPU 경합으로 수치가 오염되므로 단독 실행.
- 재현: `python3 verify_recurrent.py --rule <linear|dla|titans> --stress --bench`



# 긴 시퀀스 학습 — chunked O(L)로 OOM 없이

naive 병렬( $O(L^2)$ )으로는 불가능하던 긴 컨텍스트 학습을 **chunked linear attention( $O(L)$ )**으로 가능케 함. 학습 경로( `grok_tune.py --chunked , model.forward_chunked(return_aux=True)` )를 chunked로 전환. 4090 실측.

## 1. 학습 메모리 — chunked가 길수록 압도적 (forward+backward, MQAR)

| seq_len         | naive 학습 peak | chunked 학습 peak           | 비고          |
|-----------------|---------------|---------------------------|-------------|
| 2048 (B=8,h=2)  | 4026 MB       | <b>2261 MB</b> (~1.8× 적음) | 둘 다 가능(소배치) |
| 8192 (forward)  | 8827 MB       | <b>220 MB</b> (~40×)      | 격차가 길어로 확대  |
| 16384 (forward) | <b>OOM</b>    | 321 MB                    | naive 불가    |

→ 길이가 길수록 naive는  $O(L^2)$ 로 폭증/OOM, chunked는  $O(L)$ . **긴 시퀀스 학습은 chunked라야 가능.**

### 1b. 4090 최대 학습 seq\_len (batch=1, d256/L4/h2, chunk=256)

| seq_len              | peak            | step      |
|----------------------|-----------------|-----------|
| 32768                | 4.14 GB         | 0.5s      |
| 65536                | 8.21 GB         | 1.6s      |
| <b>131072 (128K)</b> | <b>16.32 GB</b> | 5.4s ← 최대 |
| 262144 (256K)        | OOM             | —         |

완벽한  $O(L)$ : 길이 2× → 메모리 2×(≈0.125MB/토큰). **단일 4090에서 128K 토큰 컨텍스트 학습 가능** (naive  $O(L^2)$ 면 16384에서 OOM → chunked로 8×+ 더 길게). 재현: `maxseq.py` .

## 2. 긴 컨텍스트 MQAR 학습 결과 (kv 앞쪽, query 말단, 사이 수백~수천 pad)

| 설정                       | 변형     | final recall | hard  | 학습시간    | OOM  |
|--------------------------|--------|--------------|-------|---------|------|
| seq=512, pairs=4, h=2    | linear | 0.348        | 0.264 | 126s/5k | ✅ 없음 |
| seq=512                  | dla    | 0.327        | 0.267 | 152s/5k | ✅ 없음 |
| seq=512                  | titans | 0.340        | 0.269 | 157s/5k | ✅ 없음 |
| <b>seq=2048, pairs=4</b> | linear | <b>0.349</b> | 0.267 | 112s/2k | ✅ 없음 |

chance≈0.062. 위는 5k 스텝 기준 ~0.35 부분해. **충분히 학습하면 grok한다(아래).**

titans 긴 컨텍스트 정확도 끌어올리기 (dh=128, chunked, 25k 스텝) ☆

| 설정                    | 변형              | final recall     | grok@ | 학습시간 |
|-----------------------|-----------------|------------------|-------|------|
| seq=512, pairs=4, h=2 | titans_residual | 1.00 (hard 0.62) | 15000 | 792s |

곡선: ~step14000까지 0.33 정체 → 15000에서 급점프(0.60→0.79→0.94→0.99→1.0). → 16× 긴 컨텍스트 (seq=512)에서 완벽 회상(1.0) + 추론 O(1) 상수메모리(132KB). 대가는 학습 컴퓨트 (grok@15k)뿐. "상수메모리 = 저 품질"이 아님을 긴 컨텍스트에서 실증(IMPACT.md §4).

3. 해석 (정직)

- ✅ 엔지니어링 목표 달성: seq=32(초기) → 2048(64×) 컨텍스트를 OOM 없이 학습. chunked가 핵심.
- ⚠️ 회상 품질은 부분해: 긴 컨텍스트 다중키 정확회상은 5k 스텝 내 grok 점프 안 함. 원인: (a) 선형/감쇠 계열의 다중키 회상 한계(GROKING.md), (b) 긴 pad gap이 회상 난이도↑, (c) dla/titans는 감쇠가 장거리 신호를 약화 (floor 0.9라도 수천 스텝 누적 시 감쇠). → 더 많은 스텝·용량(dh↑)·낮은 감쇠가 필요(후속). swla(softmax)는 chunked 비대상(윈도우).

결론

"더 긴 시퀀스 학습"은 가능해졌다 — chunked O(L)로 64× 긴 컨텍스트를 OOM 없이 학습(메모리 1.8~40× 절감). 회상 품질의 grok은 추가 컴퓨트가 필요(데이터 규모/예산 의존) — 정직한 현 상태.

재현

```
python3 grok_tune.py --chunked --seq-len 2048 --pairs 4 --batch 8 --n-heads 2 \
  --variants linear_residual --steps 2000
```

# grokking 튜닝 — 8변형 전부 회상 학습 + 아키텍처별 용량 천장

**결론:** 8변형 모두 MQAR 연관회상을 학습한다(>chance). 단 정확회상(near-1.0)을 grok하는 난이도는 아키텍처 메커니즘에 따라 크게 다르다. softmax·영속메모리·게이팅(감쇠+용량)이 있으면 다중키 회상을 grok하고, \*\*순수 선형 어텐션(감쇠·게이트·선택 전무)\*\*은 다중키에서 천장(~0.5)에 갇힌다.

검증 환경: RTX 4090, 실학습. 도구: `grok_tune.py` . 과제: MQAR(vocab32). 지표: query 위치 recall. grokking 가속 레버(Power et al.): weight decay · 충분한 스텝 · warmup/cosine · (선형계열) 상태 용량.

## 종합 결과 (2키 회상, chance≈0.062)

| 변형              | 최고 recall    | grok 점프  | 그것을 가능케 한 것                |
|-----------------|--------------|----------|----------------------------|
| swla_residual   | 1.00         | step 2k  | softmax 어텐션                |
| titans_ssc      | 1.00         | 4k       | 영속메모리 + top_k 선택           |
| linear_grm      | 0.93         | 5.5k     | head 게이팅                   |
| dla_residual    | 1.00         | 8k       | 데이터 의존 감쇠 + 상태 용량↑(dh=128) |
| titans_residual | 1.00         | 13k      | 영속메모리(느리지만 grok)           |
| linear_ssc      | 0.58         | ~20k 진행형 | top_k 선택(매우 느림)            |
| linear_residual | 0.54 (2키 천장) | 없음       | — (순수 선형)                  |
| linear_soup     | 0.50 (2키 천장) | 없음       | — (순수 선형)                  |

## 1키 회상(자명)에서는 전부 1.0

`linear_residual` · `linear_soup` 도 pairs=1에서는 즉시 1.0(step 1k). → 회상 자체는 학습 가능하며, 막히는 것은 다중키 정확회상 용량이다. 천장은 "2키 중 1키만 안정 회상"(=0.5)으로 나타난다.

## 긴 컨텍스트(seq=512, 16×)에서도 titans는 1.0 grok ★

titans\_residual을 seq=512(16× 긴 컨텍스트), dh=128, chunked, 25k 스텝으로 학습: recall 1.0(hard 0.62), grok@15000 (step14k까지 0.33 정체 → 0.60→0.79→0.94→1.0 급점프). → 긴 컨텍스트에서도 완벽 회상 + 추론 O(1) 상수메모리 동시 달성. 대가는 학습 컴퓨트(LONGSEQ.md).

## 시도한 레버와 효과

| 레버                                    | 효과                                            |
|---------------------------------------|-----------------------------------------------|
| 쉬운 과제(pairs 8→4→2)                    | 약한 변형도 학습 시작(부분해 0.34→0.53)                   |
| 스텝 ↑ (6k→20k)                         | titans_residual을 13k에서 grok시킴                 |
| weight decay ↑ (0.1→0.5)              | 순수 선형엔 무효(천장 불변)                              |
| 상태 용량 ↑ (head 8→2, dh 32→128, 상태 16×) | dla_residual을 grok시킴(선형 어텐션 회상 용량 ∝ 상태 크기 입증) |

## 해석 (AutoResearch 관점)

이것이 본 노드의 존재 이유를 정확히 보여준다 — "이 데이터의 회상 난이도에 맞는 (표현→기억) 구성"을 자동 탐색:

- 회상 요구가 크면 → softmax(swla)/영속메모리(titans)/게이팅(grm)/감쇠+큰상태(dla, head↓) 선택.
- 순수 선형(linear+residual/soup)은 효율은 최고지만 다중키 정확회상엔 부적합 → 효율↔정확도 트레이드오프.
- 이 천장은 선형 어텐션의 알려진 MQAR 한계(Zoology)와 일치. 스텝·wd로는 못 뚫고 **용량**이 유효 레버.

## 재현

```
python3 grok_tune.py --variants all --steps 8000 --wd 0.1 --pairs 2 --vocab 32 --seq-len 16
python3 grok_tune.py --variants dla_residual --steps 12000 --pairs 2 --n-heads 2 #
용량↑로 grok
python3 grok_tune.py --variants linear_residual,linear_soup --pairs 1 #
1키는 전부 1.0
```

# vetted growing-memory 구현 축별 검증 (정확 구현 먼저)

"정확 구현 먼저 → autoresearch 연결" 원칙으로, flash-linear-attention( fla ) + titans-pytorch의 검증된 모듈을 단일 고정 LayerNorm 스케폴드에 끼워 같은 MQAR로 검증. 재현: packages/growing-memory/fla\_validate.py .

## 결과 (MQAR vocab32/pairs2/L64, dim128, 4k step, 4090, chance≈0.062)

| 구현                   | recall | grok   | 시간   | base_rule 매핑        |
|----------------------|--------|--------|------|---------------------|
| deltanet (fla)       | 1.0    | 즉시     | 41s  | delta rule          |
| gated_deltanet (fla) | 1.0    | 즉시     | 120s | Titans류(게이트+델타)     |
| retention (fla)      | 1.0    | @3000  | 29s  | retention(스칼라 감쇠)   |
| titans (lucidrains)  | 1.0    | @3000  | 113s | Titans 논문           |
| linear (fla)         | 0.54   | 부분     | 23s  | 순수 선형(회상 약점)        |
| gla/dla (fla)        | 0.06   | chance | 21s  | GLA(게이트) — 추가 튜닝 필요 |

## 해석

- 4/6 vetted 구현이 정확회상 1.0 — 빠르고 완벽(특히 deltanet: O(1) 상태 + 1.0 + 41초 최고).
- 내 from-scratch 근사(model.py의 linear/dla/titans)는 부분해/느린 grok였으나, vetted 구현은 즉시 1.0.
- 순수 선형(fla linear)은 0.54로 회상 약점 재확인(이론과 일치). GLA는 chance — 게이트 초기화/스텝 추가 필요 (별도 튜닝 대상).
- 검증된 정답 셋: deltanet, gated\_deltanet, retention, titans → 이들을 autoresearch base\_rule로 연결.

## autoresearch 연결 (검증된 구현을 base\_rule로)

검증 통과 구현을 real/model.py 의 MemoryMixer에 base\_rule 로 연결(ext\_mem):

deltanet/gated\_deltanet/retention/gla/linear\_fla (fla) + titans\_real (lucidrains). 스케폴드는 검증과 동일하게 LayerNorm으로 정렬(기존 RMSNorm은 grok이 늦었음). fla 커널은 bf16 autocast.

연결 후 autoresearch 하니스(GrowingMemoryModel+chunked, seq64/pairs2/4k) 실행: | base\_rule | recall | 비교 | |---|---|---| | deltanet | 1.0 (grok@1000, hard 0.97) | end-to-end 연결 검증  | | retention | 0.55 | 클린 스케폴드는 1.0이나 풀 하니스는 4k 내 grok 안 됨(late-grokker, 민감) |

→ deltanet으로 "정확 구현 → autoresearch 연결" 입증(1.0 end-to-end). retention은 클린 스케폴드 검증 1.0이나 풀 하니스 정렬은 추가 폴리시 필요(weight tying/SSL front/스텝). 스위치에는 deltanet 등 검증·연결된 base\_rule을 사용. 재현: grok\_tune.py --variants deltanet,retention,titans\_real,... .

## 교훈 적용

---

- 동일 고정 스캐폴드(LayerNorm) + 같은 시드/평가셋 → 공정 비교(이전 RMSNorm 스캐폴드 혼재 문제 제거).
- from-scratch보다 **vetted 라이브러리 채택**이 정확·빠름(이번 세션 핵심 교훈).

# Titans — 이름 출처, 정확 구현, 정직한 검증 결과

## 이름 출처 (제가 만든 게 아님)

- 설계 문서(사용자 제공) §3.1의 `base_rule` 후보에 `titans` 가 이미 있었음.
- 원 출처: Google Research 논문 "Titans: Learning to Memorize at Test Time" (Behrouz, Zhong, Mirrokni — arXiv:2501.00663, 2025). 어텐션 + 테스트시점 학습 신경 장기기억.

## 논문 핵심 식

|                                                           |                                            |
|-----------------------------------------------------------|--------------------------------------------|
| $\ell(W; x_t) =   W \cdot k_t - v_t  ^2$                  | (연관 손실; $k=xW_K, v=xW_V, q=xW_Q$ )         |
| $\nabla \ell = (W \cdot k_t - v_t) \cdot k_t^T$           | (행렬 메모리 기울기)                               |
| $S_t = \eta_t \cdot S_{t-1} - \theta_t \cdot \nabla \ell$ | (모멘텀: 과거+현재 surprise)                      |
| $W_t = (1 - \alpha_t) \cdot W_{t-1} + S_t$                | ( $\alpha_t$ : 데이터 의존 망각/weight-decay 게이트) |
| $y_t = W_t \cdot q_t$                                     | (현재 메모리로 읽기)                               |

$\eta_t$  모멘텀,  $\theta_t$  학습률,  $\alpha_t$  망각(모두 데이터 의존). 추론 상태  $(W,S)=O(d^2)$  상수  $\rightarrow$  상수메모리 추론.

## 세 가지 "titans" 구현과 실측 (정직)

| 구현                                             | 무엇                    | MQAR recall(vocab32/pairs2/L64, 4090) |
|------------------------------------------------|-----------------------|---------------------------------------|
| <code>base_rule="titans"</code> (model.py)     | dla 감쇠 + 정적 영속읽기 (근사) | 부분~1.0(grok, 학습 많이) — 단 논문 메커니즘 아님    |
| <code>titans_exact.py</code> (우리 from-scratch) | 논문 식 직접 재현            | 0.06 = chance (학습 실패) ⚠️              |
| lucidrains <code>titans-pytorch</code>         | 검증된 정확 구현             | 0.98 ✅                                |

$\rightarrow$  결론: 자체 정확 구현(titans\_exact)은 chance로 실패. 안정화(키/쿼리 L2정규화+출력 norm)로 NaN(0.0) $\rightarrow$ chance(0.06)까지 갔으나 학습은 안 됨(창크병렬·정확기울기·메모리MLP 등 미충족). 정확한 Titans는 lucidrains `titans-pytorch` 를 채택( `growing_memory.titans_backend` ).

## 사용 (정확 구현)

```
pip install "growing-memory-pytorch[titans]" # titans-pytorch 포함
```

```
from growing_memory.titans_backend import neural_memory, available
assert available()
mem = neural_memory(dim=384, chunk_size=64) # lucidrains NeuralMemory
retrieved, state = mem(seq) # [B,L,d]
```

## AutoResearch 통합 (base\_rule="titans\_real")

검증된 lucidrains NeuralMemory를 노드의 믹서로 연결 — `real/model.py` 의 MemoryMixer가 `base_rule="titans_real"` 이면 `titans_pytorch.NeuralMemory` 를 사용(aggregation 우회). `grok_tune` VARIANTS에 `titans_real` 추가 → 스왑/학습에 진짜 Titans 포함.

### recall 차이의 진짜 원인 — 통합 버그(chunk\_size), 난이도 아님 (정직 정정)

처음엔 "통합 0.33 vs 교차검증 0.98"을 난이도 차이로 설명했으나 틀렸다. 변수 격리로 규명:

1. 같은 쉬운 조건(seq=64/pairs2)인데 하니스만 바뀌도 0.98→0.32 → 난이도 아님, 스캐폴드/통합 문제.
2. weight tying 해제 → 무효(0.31→0.31). 진짜 원인은 lucidrains `chunk_size` :

| chunk_size (seq=64) | recall(3k) |
|---------------------|------------|
| 16                  | 0.521      |
| 32                  | 0.520      |
| 64 (내 통합 버그)        | 0.314      |

내 통합이 `chunk_size=max(16, segment_len)=64` 로 뒤서 `seq=64`면 전체가 1청크 → 메모리 갱신 1회뿐 → 회상 붕괴. 수정: `chunk_size=min(32, segment_len)` (시퀀스 내 여러 번 갱신).

수정 후 본 런(seq=64/pairs2, 8k step): recall 0.31 → 0.55 (수정 효과 확인). 단 8k에서도 grok 점프 없이 0.55 정체 — 우리 GrowingMemoryModel 스캐폴드는 grok이 늦다(우리 titans도 13~15k 걸림; titans\_compare 스캐폴드는 chunk32에서 3k에 0.98 grok). 남은 차이는 RMSNorm vs LayerNorm + SSL front(weight tying은 배제됨). → 0.9+는 (a) 더 많은 스텝(~15~20k) 또는 (b) 스캐폴드 정렬 필요. 정직: 본 런은 0.9+ 미달(0.55), 모듈 자체는 vetted 스캐폴드에서 0.98 확인됨.

→ 교훈: 통합 시 라이브러리 하이퍼파라미터(chunk\_size)를 잘못 매핑하면 성능이 무너진다. 실행:

`~/gm_venv/bin/python grok_tune.py --chunked --variants titans_real ...` (titans-pytorch 필요).

## 교훈

사용자 지적("titans는 네가 만든 거냐?")이 정확했다. 이름은 Google 논문 차용, 자체 구현은 근사/실패였다. "정확 구현을 하거나 검색하거나" 중 검색·채택(lucidraings)이 정답이었음. 재현: `packages/growing-memory/titans_compare.py`.



# HC-SR04 실 센서 데이터 — 드리프트/이상 탐지

실 초음파 거리센서(HC-SR04) 저주파 데이터로 설계 §5/§11 "예측오차 이상탐지"를 실현·검증.

## 데이터셋

- 출처: APMonitor DDE — [https://apmonitor.com/dde/uploads/Main/dist\\_hcsr04.csv](https://apmonitor.com/dde/uploads/Main/dist_hcsr04.csv)
- 로컬: `data/hcsr04/dist_hcsr04.csv` (900행, Time, Dist(설정거리 inch), Measured(원시 에코 값) )
- 9개 설정거리(0~35 inch)에서 각 ~100샘플. Measured는 inch가 아닌 원시 카운트(감도 24.2 raw/inch).

## 데이터 특성 (분석)

- 설정거리 ↔ Measured 거의 완벽 선형:  $\text{Dist} = 0.0413 \cdot \text{Measured} - 0.39$ ,  $R^2=0.998$ , RMSE 15mm.
- 원거리 드리프트: set=35가 시간상 두 번(초기 t=5, 후기 t=57) 측정됨 → 평균 875.7 → 858.8 ( $\Delta \approx -15 \sim -17$ ). 원거리 노이즈( $\sigma \approx 9.5$ )가 다른 거리( $\sigma \approx 2$ )보다 큼(약신호 반사).

## 탐지기 ( `hcsr04_anomaly.py` , `stdlib` )

데이터 규모(900행·저복잡도)에 맞춰 딥모델 대신 견고한 통계 탐지기 — 정직한 도구 선택.

- 세그먼트: 설정거리 변화 기준 연속 구간(같은 거리 재방문 → 별도 세그먼트).
- 정상 baseline: 거리별 첫 세그먼트의 robust 중심(median) +  $\sigma$ (MAD/std/바닥값 2.0 중 최대).
- 점 이상:  $z=(x-\text{median})/\sigma$ ,  $|z|>4 \rightarrow$  플래그.
- 드리프트: 후속 세그먼트 평균 이동을 표준오차( $SE=\sigma/\sqrt{n}$ )로 검정 — n개 평균의 작은 이동도 유의할 수 있음(점  $\sigma$ 로 보면 놓침).

## 결과 (실측)

| 항목                    | 값                                                        |
|-----------------------|----------------------------------------------------------|
| 세그먼트                  | 9 (설정거리 9개)                                              |
| 실데이터 점 이상             | 0 (정상, 오탐 없음)                                            |
| 실데이터 드리프트             | set=35 후기: $\Delta=-15.2$ , $-22.8$ SE 탐지 (나머지 7거리 오탐 0) |
| 검증(주입 20개: 스파이크·드롭아웃) | recall 0.95, 오탐 0                                        |

→ 실제 센서 드리프트를 정확히 포착하고, 합성 이상치도 95% 검출. 점이상= $\sigma$  / 드리프트=SE 분리가 핵심.

## 노드 연결

---

- 이 탐지기는 §5 프록시 "예측오차 이상탐지"의 실테이터 인스턴스. NAS 도커 서버에 HC-SR04 샤드를 주입하면 (같은 인터페이스) 실 센서 스트림으로 노드를 돌릴 수 있다.
- 모델링 실현성 결론: **단순 보정은 선형회귀로 충분(딥모델 과함)**, 가치 있는 ML 과제는 **드리프트/이상 탐지**(본 문서)와 **다음값 예측**(short\_horizon 프록시).

## 재현

---

```
curl -o data/hcsr04/dist_hcsr04.csv  
      https://apmonitor.com/dde/uploads/Main/dist_hcsr04.csv  
python3 tutorial/autoresearch/hcsr04_anomaly.py
```

# 실험 배터리 결과 (작은 실험 전부 실행)

실행 환경: RTX 4090 24GB ( martin@linux-builder ), torch 2.10.0+cu128, cuda\_op\_ok: True . 재  
현: scripts/run\_experiments.sh (mock 배터리) · run.py run --config  
config/run\_real.yaml (실물).

## A. 실물(real) 백엔드 — 실제 PyTorch 학습 결과 ★

growing-memory-pytorch 업스트림이 없어 설계 4축 + SSL을 torch로 직접 구현( autoresearch/real/ ) 하  
고, 실제 MQAR 연관회상 과제로 4090에서 학습/평가. backend= real .

실물 ASHA (rungs=[400,1000,2500] 실제 학습 스텝, eta=3, max\_trials=9, MQAR 4쌍/vocab40):

| trial | config                 | proxy(recall) | full  | cost(step) | 승급          |
|-------|------------------------|---------------|-------|------------|-------------|
| t0006 | dla+grm d256L4 seg32   | 0.327         | 0.249 | 2500       | → top rung  |
| t0002 | swla+residual d128L2   | 0.309         | -     | 1000       | → rung1     |
| t0005 | titans+residual d256L2 | 0.308         | -     | 1000       | → rung1     |
| t0008 | swla+grm d256L4        | 0.299         | -     | 1000       | → rung1     |
| t0004 | linear+ssc d256L2      | 0.249         | -     | 400        | rung0       |
| t0003 | titans+soup            | 0.094         | -     | 400        | rung0(조기중단) |
| t0007 | linear+soup            | 0.034         | -     | 400        | rung0(조기중단) |
| t0001 | titans+ssc             | 0.000         | -     | 400        | rung0(조기중단) |

chance recall  $\approx$  0.05. backend=real, jobs\_run=14, gate 탈락 0.

### 발견 (실물)

1. **실학습 파이프라인 동작 확인.** build(cfg)→실제 torch 모델→AdamW 학습(2500스텝)→MQAR recall 평가까지 4090에서 end-to-end. best= dla+grm recall 0.249(chance 0.05의 ~5배).
2. **ASHA 조기중단이 실측으로 작동.** rung0(400스텝)에서 proxy 낮은 config(0.0~0.09)는 승급 차단, 상위만 1000→2500으로 승급. 자원 낭비 없이 학습되는 config로 집중.
3. **스왑이 "작동 조합"을 발견 — autoresearch의 본질.** 단일 고정 config 스모크에선 dla/titans/ssc가 chance였으나, 탐색은 dla+grm · titans+residual · linear+ssc 등 학습되는 조합을 찾아냄. 조합 (aggregation×base\_rule×하이퍼파라미터) 의존성이 크다는 것을 실측으로 확인.
4. **MQAR grokking 한계.** recall은 급격 상전이라 일부 조합(soup, titans+ssc)은 현재 예산에서 임계 미달. 예산↑/튜닝으로 보정 대상( real/README.md ). 선형/소프트맥스 어텐션이 회상에 유리 (Zoology 경향)와 일치.

실물 연결: 머신에서 `export GROWING_MEMORY_HOME=<repo>` 시 업스트림 우선, 없으면 이 참조 구현, torch 없으면 mock — 인터페이스 무변경. 강제 mock: `AR_FORCE MOCK=1` .

## B. mock 백엔드 — 루프 폐쇄/탐색 로직 검증

백엔드 = mock(torch 없이 stdlib, 합성 신호). 탐색·게이트·리더보드·ratchet 로직 검증용.

### 스윙 실험 (ASHA, rungs=[1000,4000,16000], eta=4, max\_trials=24)

| 실험         | proxy              | 표현축               | jobs | gate 탈락 | top rung 도달 | rank_corr   | best full     | best 계열         |
|------------|--------------------|-------------------|------|---------|-------------|-------------|---------------|-----------------|
| E1         | factory_mqar       | full(SSL)         | 35   | 10      | 3           | 0.50        | <b>0.7812</b> | titans+ssc      |
| E2         | short_horizon_pred | full(SSL)         | 37   | 13      | 3           | <b>1.00</b> | 0.7812        | titans+ssc      |
| E3         | factory_mqar       | <b>T0(SSL 고정)</b> | 35   | 6       | 2           | 1.00        | <b>0.6018</b> | titans+residual |
| E4 (seed1) | factory_mqar       | full              | 31   | 4       | 1           | -           | 0.7818        | titans+ssc      |
| E4 (seed2) | factory_mqar       | full              | 32   | 5       | 1           | -           | 0.7562        | titans+ssc      |
| E4 (seed3) | factory_mqar       | full              | 31   | 4       | 1           | -           | 0.7749        | dla+ssc         |

### 발견

- 표현축(SSL)이 가장 큰 레버 — 실측 확인.** 풀공간 best(0.78) vs T0(SSL 고정) best(0.60), 절대 +0.18. 설계 §3.2의 "positive\_pair가 가장 큰 레버" 가설과 일치.
- best 계열 수렴 안정성.** 시드 1·2·3에서 best가 모두 `*+ssc` 기억축 + `titans/dla` base로 수렴(full 0.75~0.78). 작은 모델 + 효율 RNN으로 싸게 많이 시도해도 winner 계열이 일관됨.
- 프록시 신뢰도는 과제 의존.** `short_horizon_pred`는 rank\_corr=1.0(프록시↔full 완벽 일치), `factory_mqar`는 0.5(medium). → §5대로 프록시는 주기적 보정 필요.
- ASHA 한계(관측).** max\_trials=24·eta=4에서는 top rung 생존이 1~3개에 그쳐, 시드에 따라 rank\_corr 계산 불가(≥2 필요). 보정안: max\_trials↑ 또는 eta↓로 상위 rung 생존 수 확보.

## Skill / 원형 / 인터페이스 점검

| 항목                                         | 결과                                                                                                                                            |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| validity-gate (space 300 샘플)               | 통과율 80.7%, 탈락 전부 <code>d_model % n_heads</code> (게이트 정상)                                                                                      |
| leaderboard-analysis (E1)                  | rank_corr 0.5 → proxy_trust=medium, best export_safe=True                                                                                     |
| lab ratchet (karpathy 원형, snapshot 30iter) | seed0: best <b>1.0228</b> (keep 11 / revert 20), seed1: best <b>1.0531</b> (keep 9 / revert 22) — val_bpb 단조 개선                               |
| MCP selftest                               | 7 도구 정상( <code>leaderboard_top/summary/get/write</code> , <code>nas_list_shards</code> , <code>run_status</code> , <code>export_best</code> ) |
| export (E1 best)                           | 번들 생성 OK, encoder=vjepa, full=0.7812                                                                                                          |

## 다음 단계

- `GROWING_MEMORY_HOME` 연결 시 동일 배터리가 **실학습**으로 전환(mock→real). 인터페이스 무변경.
- factory\_mqar 프록시를 공장 데이터 구조에 맞춰 구체화(\$11) → rank\_corr 개선.